



RAMAIAH
Institute of Technology

RAMAIAH INSTITUTE OF TECHNOLOGY

Department of CSE(AI&ML) and CSE(CS)
(An Autonomous Institution, Affiliated to VTU)

Operating Systems

Unit 2

Subject Code: CI42

B.E Sem: IV

Credits : 4



Syllabus

Unit II

Process Management: Basic concept; Process scheduling; Operations on processes; Inter process Communication.

Threads: Overview; Multithreading models;

Process scheduling: Basic concepts, Scheduling criteria, scheduling algorithms, multiple processor scheduling, Algorithm evaluation.



The Foundation: CPU scheduling is the absolute basis of multiprogrammed operating systems.

The Objective: To have some process running at all times in order to maximize CPU utilization.

How it Works: In a simple system, when a process waits for an I/O request, the CPU sits idle and time is wasted. With multiprogramming, when one process has to wait, the OS immediately takes CPU away and gives it to another ready process.



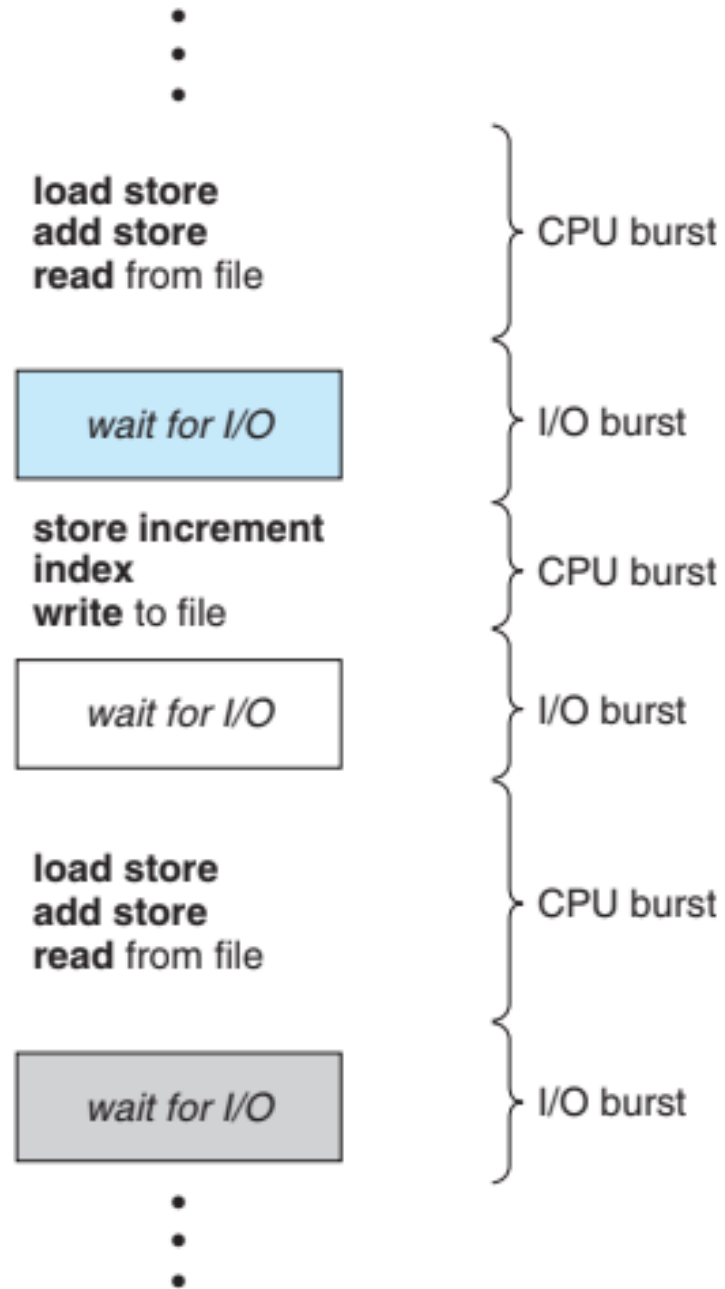
The Cycle: The success of CPU scheduling relies on an observed property: process execution consists of an alternating cycle of CPU execution and I/O wait.

Alternating States:

- Execution always begins with a CPU burst.
- This is followed by an I/O burst, then another CPU burst, and so on.
- The final CPU burst ends with a system request to terminate execution.



CPU-I/O Burst Cycle



CPU Burst Duration Distribution

Burst Durations: CPU burst durations have been measured extensively. While they vary greatly, they tend to follow a specific frequency curve (exponential or hyperexponential).

The Golden Rule of Bursts:

- There is a large number of very short CPU bursts.
- There is a small number of very long CPU bursts.

Process Types:

- **I/O-Bound Programs:** Typically have many very short CPU bursts.
- **CPU-Bound Programs:** Typically have a few very long CPU bursts.

Importance: Understanding this distribution is critical when selecting the appropriate CPU-scheduling algorithm.

CPU Burst Duration Distribution

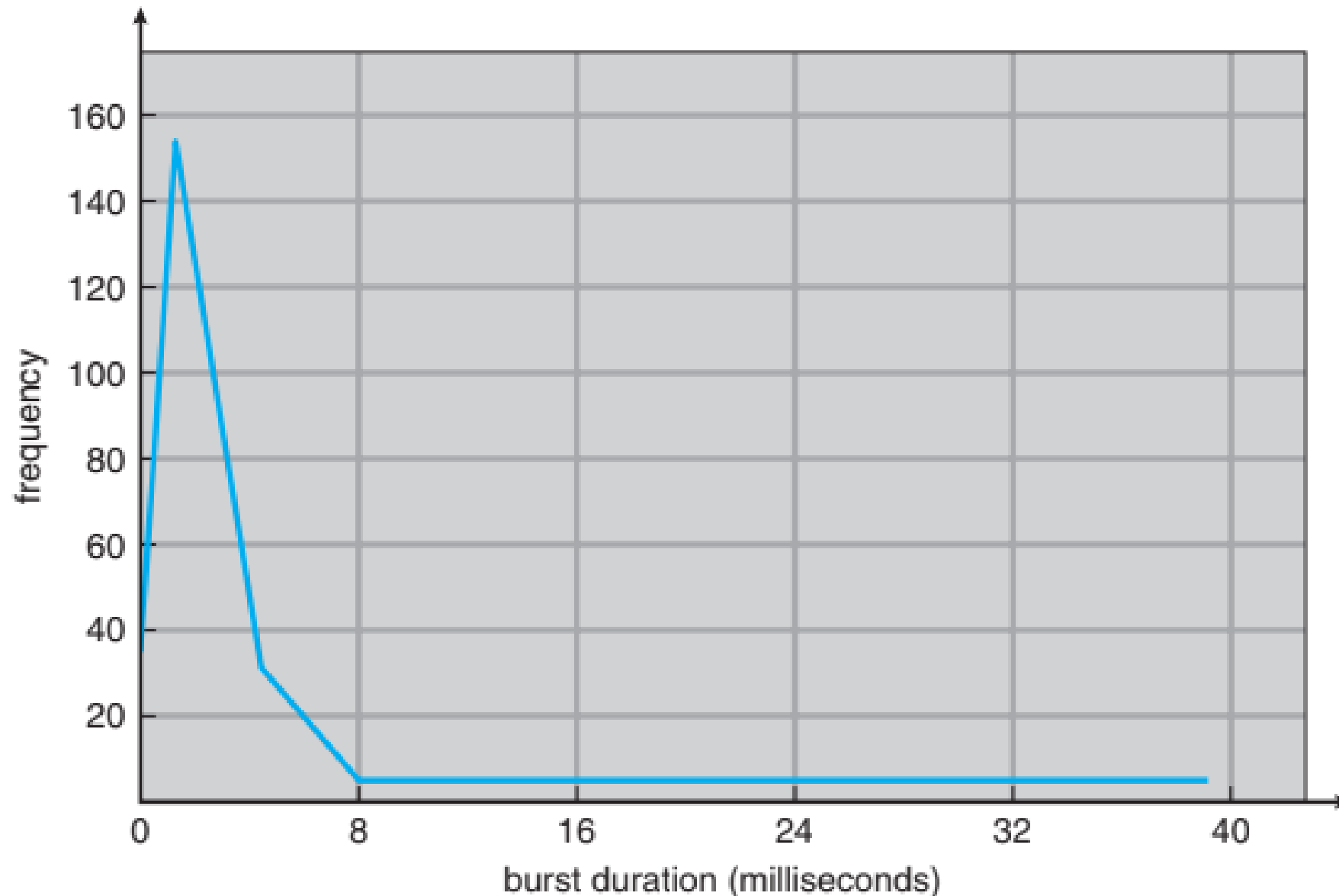


Figure 6.2 Histogram of CPU-burst durations.

CPU Scheduler

The Trigger: Whenever the CPU becomes idle, the operating system must step in and select one of the processes in the ready queue to be executed.

The Short-Term Scheduler: Also known as the CPU scheduler, this module carries out the selection. It picks a process from memory that is ready to execute and allocates the CPU to it.

The Ready Queue Structure:

- Conceptually, all processes in the ready queue are lined up waiting for a chance to run on the CPU.
- However, it is not necessarily a first-in, first-out (FIFO) queue.
- Depending on the scheduling algorithm, it can be implemented as a FIFO queue, a priority queue, a tree, or simply an unordered linked list.

Queue Contents: The records sitting in these queues are generally the Process Control Blocks (PCBs) of the processes, not the actual programs themselves.



The Four Scheduling Circumstances: CPU-scheduling decisions take place under four specific conditions:

1. When a process switches from running to waiting (e.g., making an I/O request).
2. When a process switches from running to ready (e.g., a hardware interrupt occurs).
3. When a process switches from waiting to ready (e.g., an I/O request completes).
4. When a process terminates.



Nonpreemptive (Cooperative) Scheduling:

- Scheduling takes place only under circumstances 1 and 4.
- Once the CPU is allocated to a process, the process explicitly keeps it until it voluntarily releases it (by waiting or terminating).
- **Usage:** Older operating systems like Windows 3.x and early Macintosh OS.

Preemptive Scheduling:

- The OS can forcibly step in and make scheduling choices during circumstances 2 and 3.
- **Usage:** All modern operating systems (Windows 95+, Mac OS X), but it requires special hardware like a timer to function.



The Cost of Preemption: While preemptive scheduling keeps the system responsive, it introduces major complexities.

Race Conditions:

- Preemption can cause chaos when data is shared among several processes.
- If Process A is preempted exactly in the middle of updating shared data, Process B might run and read that data while it is in a corrupted, inconsistent state.



Kernel Design Complications:

- If a process is preempted while in kernel mode (e.g., executing a system call that changes important I/O queues), it can break the operating system.
- **Solution:** Some OSs (like UNIX) simply wait for the system call to complete before allowing a context switch, though this harms real-time performance.

Interrupt Handling:

- Because interrupts can happen at any millisecond, crucial OS activities must be carefully guarded.
- OS will briefly disable interrupts at entry of crucial code sections and re-enable them at exit to prevent simultaneous access.



The Dispatcher

The Doer vs. The Decider: While the short-term scheduler selects which process runs next, the dispatcher is the module that actually gives control of the CPU to that selected process.

The Three Functions: To successfully hand over control, the dispatcher must perform three specific tasks:

- **Switching context:** Saving the state of the old process and loading the state of the new one.
- **Switching to user mode:** Moving from the kernel's privileged mode back to standard user mode to run the application.
- **Jumping to the proper location:** Finding the exact line of code where the user program previously paused and restarting it from there.



Dispatch Latency: The specific amount of time it takes for the dispatcher to stop one process and start another running.

- Because the dispatcher is invoked during every single process switch, it must be engineered to be as fast as possible to minimize this latency.

CPU Scheduling Criteria

How do we judge if a scheduling algorithm is "good"? We measure it using five specific criteria:

- **CPU Utilization:** Keeping the CPU as busy as possible. In a real system, this ranges from 40% (lightly loaded) to 90% (heavily loaded).
- **Throughput:** The number of processes that are completely executed per time unit.
- **Turnaround Time:** The total interval from the time of submission of a process to the time of its completion.
- **Waiting Time:** The sum of the periods a process spends waiting in the ready queue.
- **Response Time:** The time it takes from the submission of a request until the first response is produced (not the time it takes to output the whole response).



Scheduling Algorithms



CPU Scheduling Alg.: -

Non Preemptive

- FCFS → First-Come, First-Serve
- SJF/SJN → Shortest Job First
Shortest Job Next
- LJF/LJN → Longest Job First
Longest Job Next
- HRRN → Highest Response Ratio Next

Preemptive

- Round Robin
- SRTF/SRTN → Shortest Remaining Time First
Shortest Remaining Time Next
- LRTF/LRTN → Longest Remaining Time First
Longest Remaining Time Next
- Multilevel Queue Scheduling
- Multilevel Feedback Queue Scheduling



The Concept: By far the simplest CPU-scheduling algorithm. The process that requests the CPU first is allocated the CPU first.

Implementation: Easily managed using a standard FIFO (First-In, First-Out) queue.

- When a process enters the ready queue, its Process Control Block (PCB) is linked onto the tail of the queue.
- When the CPU is free, it is allocated to the process at the head of the queue.



Nonpreemptive: FCFS is strictly a nonpreemptive algorithm.

- Once a process gets the CPU, it keeps it until it voluntarily releases it (either by terminating or requesting I/O).
- **Drawback:** This makes FCFS particularly troublesome for time-sharing systems where users require a share of the CPU at regular intervals.

First-Come, First-Served (FCFS) Scheduling

The Major Drawback: The average waiting time under the FCFS policy is generally not minimal, and it can vary substantially depending on arrival order.

Example 1 (Long Wait):

- Processes: P1 (24ms burst), P2 (3ms burst), P3 (3ms burst).
- Arrival Order: P1, P2, P3.
- Waiting Times: P1 = 0ms, P2 = 24ms, P3 = 27ms.
- Average: $(0 + 24 + 27) / 3 = 17$ milliseconds.

Example 2 (Short Wait):

- Arrival Order: P2, P3, P1.
- Waiting Times: P2 = 0ms, P3 = 3ms, P1 = 6ms.
- Average: $(0 + 3 + 6) / 3 = 3$ milliseconds.

Conclusion: Simply changing the arrival order caused a massive reduction in the average waiting time.



First-Come, First-Served (FCFS) Scheduling

Dynamic Situations: How does FCFS perform in a real, dynamic environment with varying process types?

The Scenario: Assume one large CPU-bound process and many short I/O-bound processes.

- The CPU-bound process gets the CPU and holds it for a long time.
- Meanwhile, all I/O-bound processes quickly finish their I/O and pile up in the ready queue. **Result:** I/O devices sit totally idle.
- The CPU-bound process finally moves to I/O. The I/O-bound processes execute their tiny CPU bursts incredibly fast and rush back to the I/O queues. **Result:** The CPU sits totally idle.

The Convoy Effect: This phenomenon occurs when all other processes are forced to wait for one big process to get off the CPU.

- **Result:** Significantly lower CPU and device utilization than if the shorter processes were allowed to go first.



Shortest-Job-First (SJF) Scheduling

The Concept: The shortest-job-first (SJF) algorithm associates each process with the length of its next CPU burst. When the CPU becomes available, it is assigned to process that has the smallest next CPU burst.

Breaking Ties: If the next CPU bursts of two processes are exactly the same, standard FCFS scheduling is used to break the tie.

Provably Optimal: The SJF scheduling algorithm is mathematically provably optimal. It guarantees the minimum average waiting time for a given set of processes.

- **Why?** Moving a short process before a long one decreases the waiting time of the short process significantly more than it increases the waiting time of the long process.

Shortest-Job-First (SJF) Scheduling

The Major Flaw: The real difficulty with the SJF algorithm is knowing the length of the next CPU request before it actually happens.

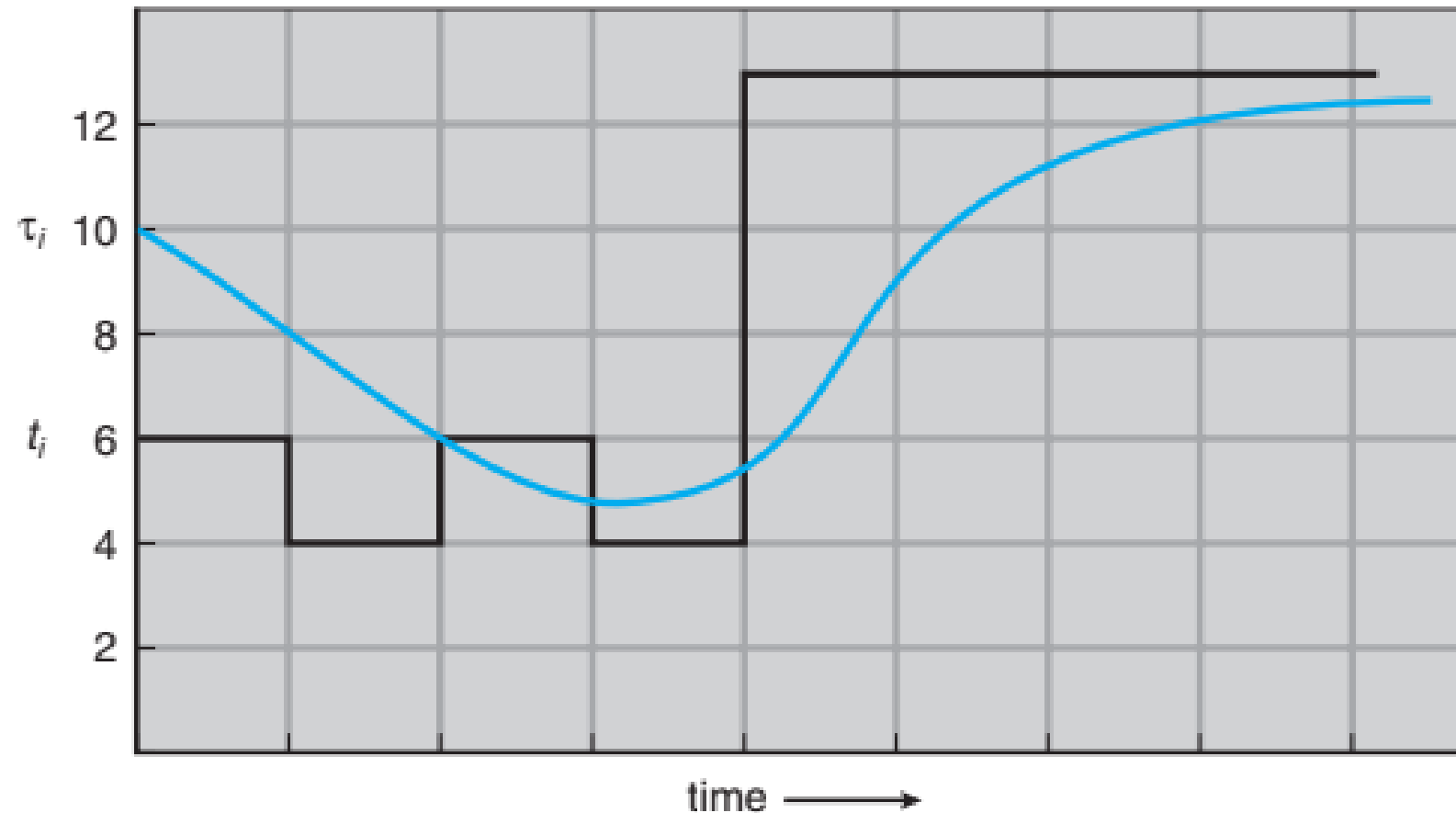
- For long-term batch systems, users can simply state their process time limits.
- For short-term CPU scheduling, there is no way to definitively know the length of the next CPU burst.

The Solution (Prediction): We must approximate SJF scheduling by predicting the value. We do this by assuming that the next CPU burst will be similar in length to the previous ones.

Exponential Averaging: The next CPU burst is predicted using a mathematical formula called an exponential average.

- It takes the actual length of the most recent CPU burst and the previously predicted burst, and weights them using a parameter (usually $1/2$) to balance recent history with past history.

Shortest-Job-First (SJF) Scheduling



CPU burst (t_i)		6	4	6	4	13	13	13	...
"guess" (τ_i)	10	8	6	6	5	9	11	12	...

Figure 6.3 Prediction of the length of the next CPU burst.

Shortest-Remaining-Time-First (Preemptive SJF)

The Arrival Problem: A choice must be made when a new process arrives at the ready queue while a previous process is still executing on the CPU.

- **The Catch:** The next CPU burst of the newly arrived process might be shorter than what is left of the currently executing process.

Shortest-Remaining-Time-First (SRTF):

- A preemptive SJF algorithm will forcibly step in and preempt the currently executing process to give the CPU to the newly arrived, shorter process.
- A nonpreemptive SJF algorithm will simply allow the currently running process to finish its CPU burst, regardless of new arrival.

Priority Scheduling

The General Concept: The SJF algorithm is actually just a special case of a broader algorithm called Priority Scheduling.

- A priority number (usually an integer) is associated with each process.
- The CPU is allocated to the process with the highest priority.

Defining Priorities:

- **Internal Priorities:** Set by the OS using measurable data (e.g., time limits, memory limits, number of open files).
- **External Priorities:** Set by criteria outside the OS (e.g., user importance, money paid, political factors).

Two Variations:

- **Preemptive:** Will forcibly kick a running process off the CPU if a newly arrived process has a higher priority.
- **Nonpreemptive:** Will simply put the new high-priority process at the very front of the ready queue.



Priority Scheduling

The Major Problem: Starvation (Indefinite Blocking):

- A priority scheduling algorithm can leave low-priority processes waiting indefinitely.
- In a heavily loaded system, a steady, endless stream of higher-priority processes can prevent a low-priority process from ever getting the CPU.
- Outcome: The low-priority process will either finally run at 2 A.M. when the system is empty, or the computer will crash and lose the process entirely.

The Solution: Aging:

- Aging involves gradually increasing the priority of processes that wait in the system for a long time.
- Example: If priorities range from 127 (lowest) to 0 (highest), the OS could decrease a waiting process's priority number by 1 every 15 minutes.
- Eventually, even a priority-127 process will age into a priority-0 process and is guaranteed to be executed.

The Concept: The round-robin (RR) scheduling algorithm is designed especially for time-sharing systems. It is very similar to FCFS scheduling, but preemption is added to enable the system to switch between processes.

The Time Quantum: The OS defines a small unit of time, called a time quantum (or time slice), which is generally from 10 to 100 milliseconds in length.



How It Works:

- The ready queue is treated as a circular FIFO queue, with new processes added to the tail.
- The CPU scheduler picks the first process, sets a timer to interrupt after exactly 1 time quantum, and dispatches it.

Two Outcomes:

- Process finishes early: If the CPU burst is less than 1 time quantum, the process voluntarily releases the CPU.
- Process takes too long: If the burst is longer than 1 quantum, the timer goes off, causes an interrupt, executes a context switch, and forcibly puts the process at the tail of the ready queue.



The Goldilocks Problem: The overall performance of the RR algorithm depends heavily on the specific size of the time quantum.

Too Large: If the time quantum is extremely large, the RR policy acts exactly the same as the FCFS policy.

Too Small: If the time quantum is extremely small (e.g., 1 millisecond), it results in a massive number of context switches, slowing down the execution of the process due to overhead.



Rules of Thumb for System Designers:

- The context-switch time (typically < 10 microseconds) should be a very small fraction of the time quantum.
- At least 80 percent of CPU bursts should be shorter than the time quantum.

Turnaround Time: The average turnaround time does not necessarily improve as the time-quantum size increases.

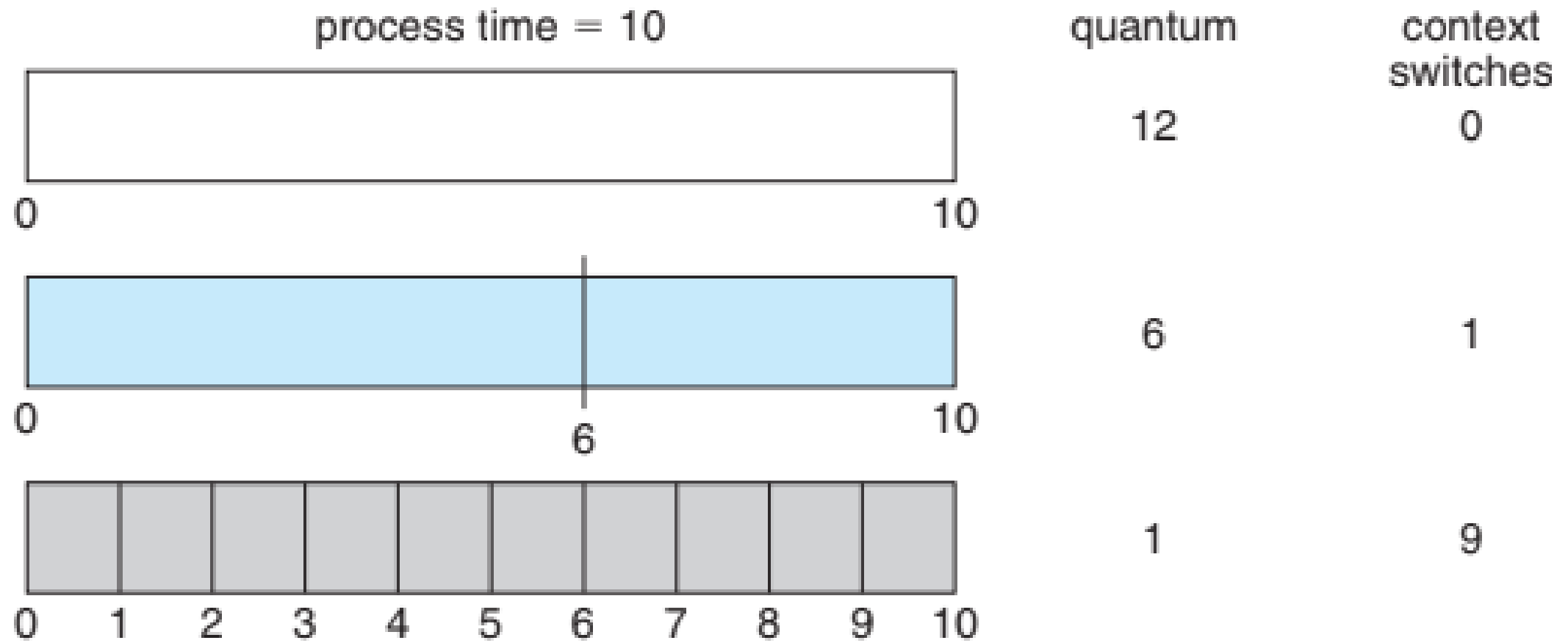


Figure 6.4 How a smaller time quantum increases context switches.

Multilevel Queue Scheduling

The Concept: Created for situations where processes can be easily classified into different groups based on their response-time requirements (e.g., foreground/interactive processes vs. background/batch processes).

Partitioning the Ready Queue: The ready queue is partitioned into several completely separate queues.

Permanent Assignment: Processes are permanently assigned to one specific queue upon entering the system, generally based on properties like memory size, priority, or process type.

Independent Algorithms: Each separate queue has its own distinct scheduling algorithm.

- Example: The foreground queue might use Round-Robin (RR) to ensure quick responses, while the background queue uses First-Come, First-Served (FCFS).



Multilevel Queue Scheduling

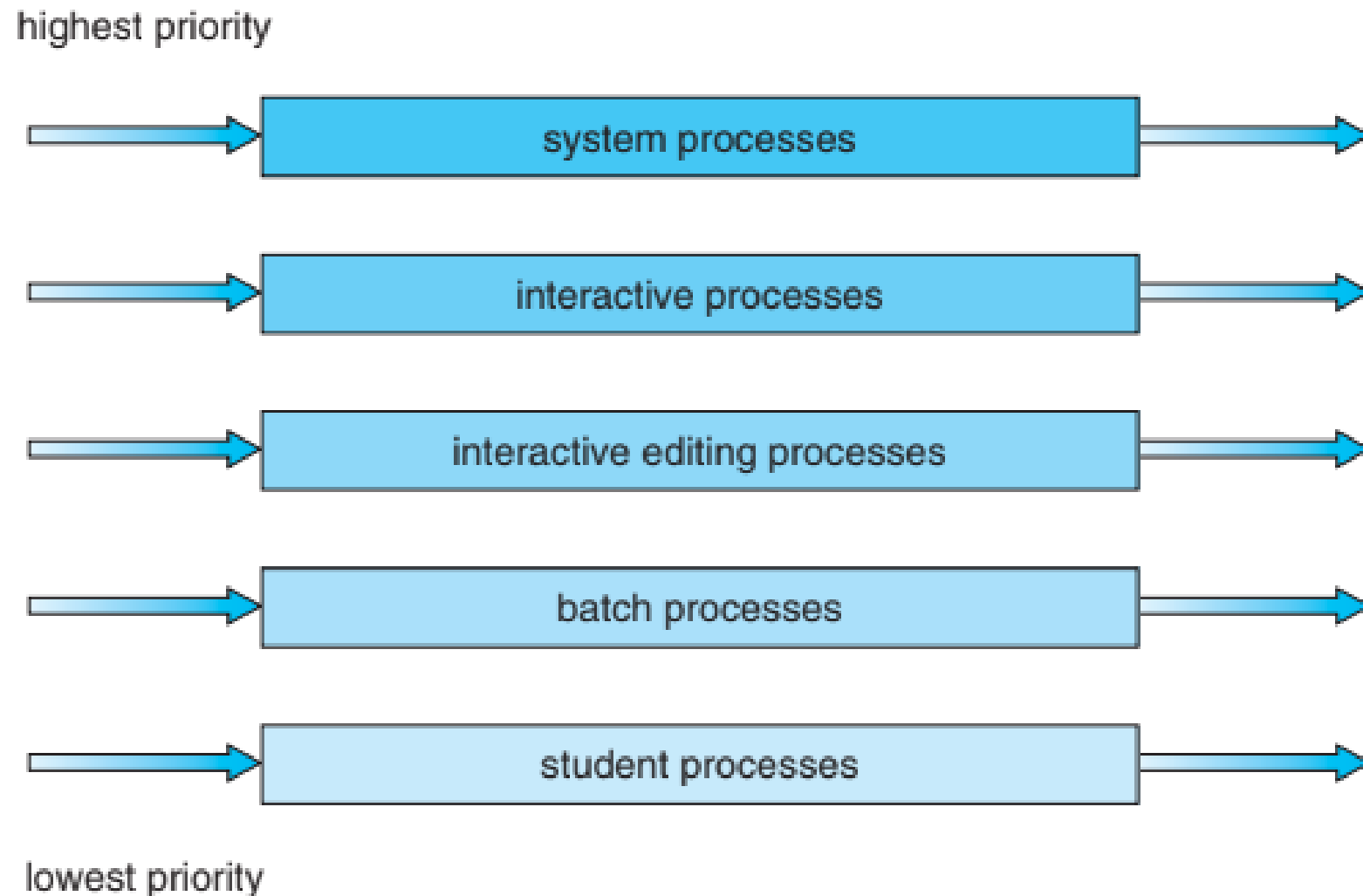


Figure 6.6 Multilevel queue scheduling.

The New Challenge: Because there are multiple queues, the OS must establish a scheduling algorithm among the queues themselves to decide which queue gets the CPU next.

Method 1: Fixed-Priority Preemptive Scheduling:

- Each queue has absolute priority over lower-priority queues.
- Example: No process in the 'Batch' queue can run unless the System, Interactive, and Editing queues are all 100% empty. If a batch process is running and an interactive process arrives, the batch process is instantly preempted.
- Major Flaw: Possibility of starvation for lower-priority queues.



Method 2: Time-Slicing Among Queues:

- Each queue gets a certain guaranteed portion of the overall CPU time, which it then schedules among its own processes.
- Example: The foreground queue gets 80% of the CPU time to run its RR algorithm, and the background queue gets the remaining 20% of CPU time to run its FCFS algorithm.



Multilevel Feedback Queue Scheduling

The Problem with Standard Queues: Normally, in a multilevel queue, processes are permanently assigned to a queue and cannot move, which has low overhead but is highly inflexible.

The Feedback Solution: The multilevel feedback queue scheduling algorithm allows a process to dynamically move between queues.

Behavior-Based Sorting: The system separates processes according to the characteristics of their CPU bursts.

- If a process uses too much CPU time, it is demoted to a lower-priority queue.
- This naturally leaves quick, I/O-bound and interactive processes in the higher-priority queues.

Preventing Starvation: A process that waits too long in a lower-priority queue may be upgraded to a higher-priority queue; this form of aging prevents starvation.



Multilevel Feedback Queue Scheduling

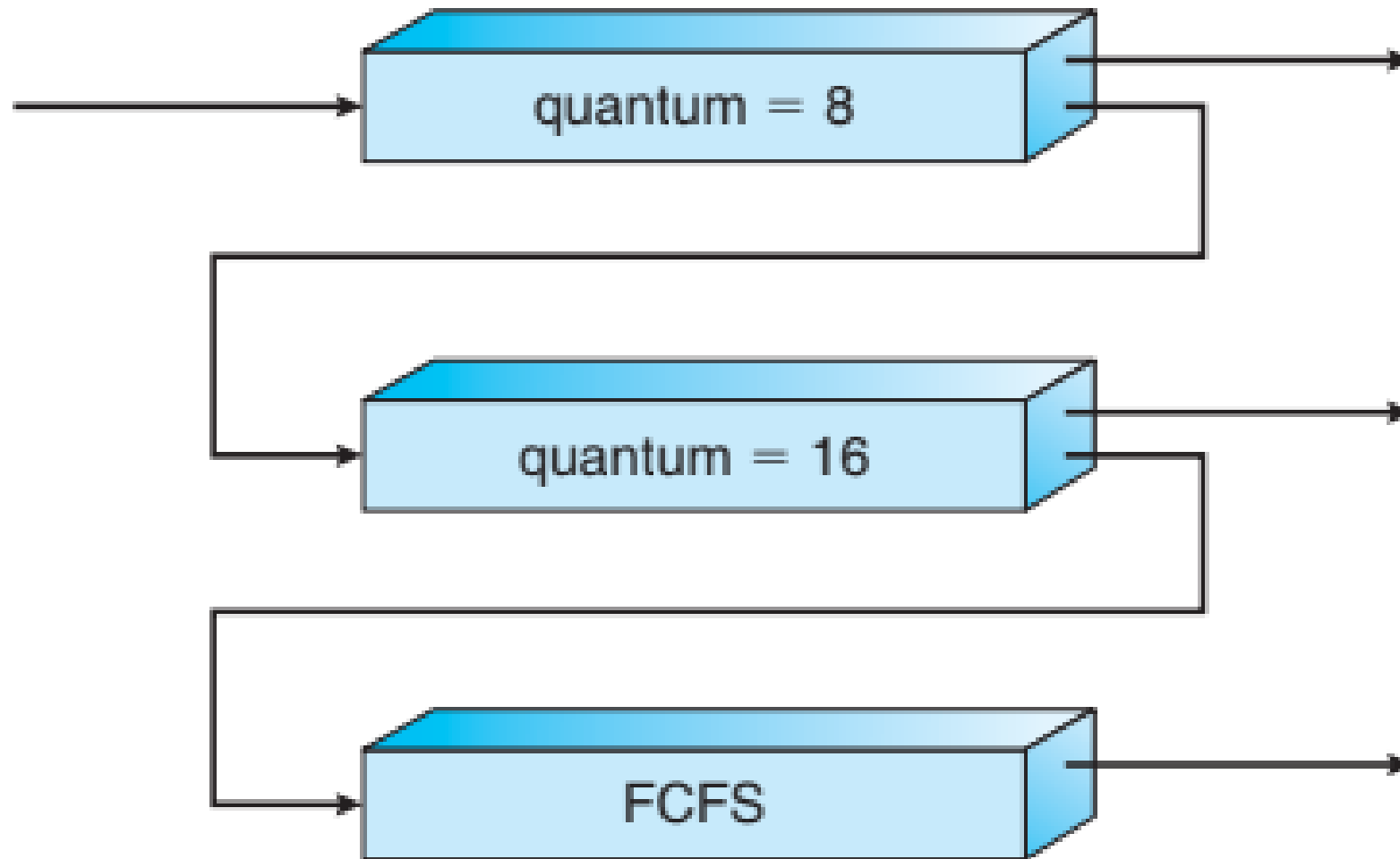


Figure 6.7 Multilevel feedback queues.



A 3-Queue Example:

- Queue 0 (Highest Priority): Uses Round-Robin with an 8ms time quantum. If a process doesn't finish in 8ms, it is preempted and moved to the tail of Queue 1.
- Queue 1 (Medium Priority): Uses Round-Robin with a 16ms quantum. If it still doesn't complete, it is demoted to Queue 2.
- Queue 2 (Lowest Priority): Uses FCFS. Processes here only run when Queues 0 and 1 are completely empty.



The 5 Defining Parameters: Building this scheduler requires choosing five specific rules:

- The number of queues.
- The scheduling algorithm for each queue.
- The method to determine when to upgrade a process.
- The method to determine when to demote a process.
- The method to determine which queue a process enters initially.

Conclusion: It is the most general and flexible CPU-scheduling algorithm, but also the most complex to configure.



Multiple-Processor Scheduling



Introduction to Multiple-Processor Scheduling

The New Paradigm: If multiple CPUs are available, load sharing becomes possible, but scheduling problems become correspondingly more complex.

Homogeneous Processors: We generally assume systems where the processors are identical (homogeneous) in functionality, meaning any available processor can be used to run any process in the queue.

Hardware Limitations: Even with identical processors, limitations exist. If a system has an I/O device attached to a private bus of one specific processor, processes needing that device must be scheduled on that specific processor.



Approach 1: Asymmetric Multiprocessing:

- A single processor (the master server) handles all scheduling decisions, I/O processing, and system activities.
- The other processors execute only standard user code.
- Advantage: Very simple to implement because only one processor accesses the system data structures, reducing the need for data sharing.



Approach 2: Symmetric Multiprocessing (SMP):

Each individual processor is entirely self-scheduling. There is no "master" processor.

Queue Structures:

- All processes may sit in one large, common ready queue.
- Alternatively, each processor may have its own separate, private queue of ready processes.

The Synchronization Challenge: Schedulers for each processor independently examine the ready queue to select a process to execute.

- Danger: If multiple processors try to access and update a common data structure, the OS must be programmed carefully.
- The system must perfectly ensure that two separate processors do not accidentally choose to schedule the exact same process, and that processes aren't lost from the queue.



The Cost of Migration: When a process runs on a specific processor, its most recently accessed data populates that processor's ultra-fast cache memory.

- If the process migrates to a new processor, the old cache is invalidated, and the new cache must be entirely repopulated. This is incredibly costly to system performance.

Processor Affinity: To avoid this massive overhead, most Symmetric Multiprocessing (SMP) systems attempt to keep a process running on the exact same processor it started on.



Two Types of Affinity:

- **Soft Affinity:** The operating system has a policy of attempting to keep a process on a single processor, but it does not guarantee it. (The process might still migrate if load balancing requires it).
- **Hard Affinity:** The system provides specific system calls (like `sched_setaffinity()` in Linux) allowing a process to strictly specify a subset of processors on which it is allowed to run.



NUMA (Non-Uniform Memory Access)

Memory Architecture Matters: The physical main-memory architecture of a system can heavily dictate processor affinity rules.

What is NUMA? Non-Uniform Memory Access occurs in systems containing combined CPU and memory boards.

- A CPU can access the memory physically located on its own board significantly faster than memory located on other boards in the system.

Blurring the Lines: To optimize this, the CPU scheduler and the memory-placement algorithms must work perfectly together.

- If a process is assigned affinity to a specific CPU, it should automatically be allocated memory on the specific board where that CPU resides.

OS Design Reality: This perfectly illustrates that the strict, solid boundaries between operating system modules (like CPU scheduling vs. Memory management) are frequently blurred to optimize hardware performance.

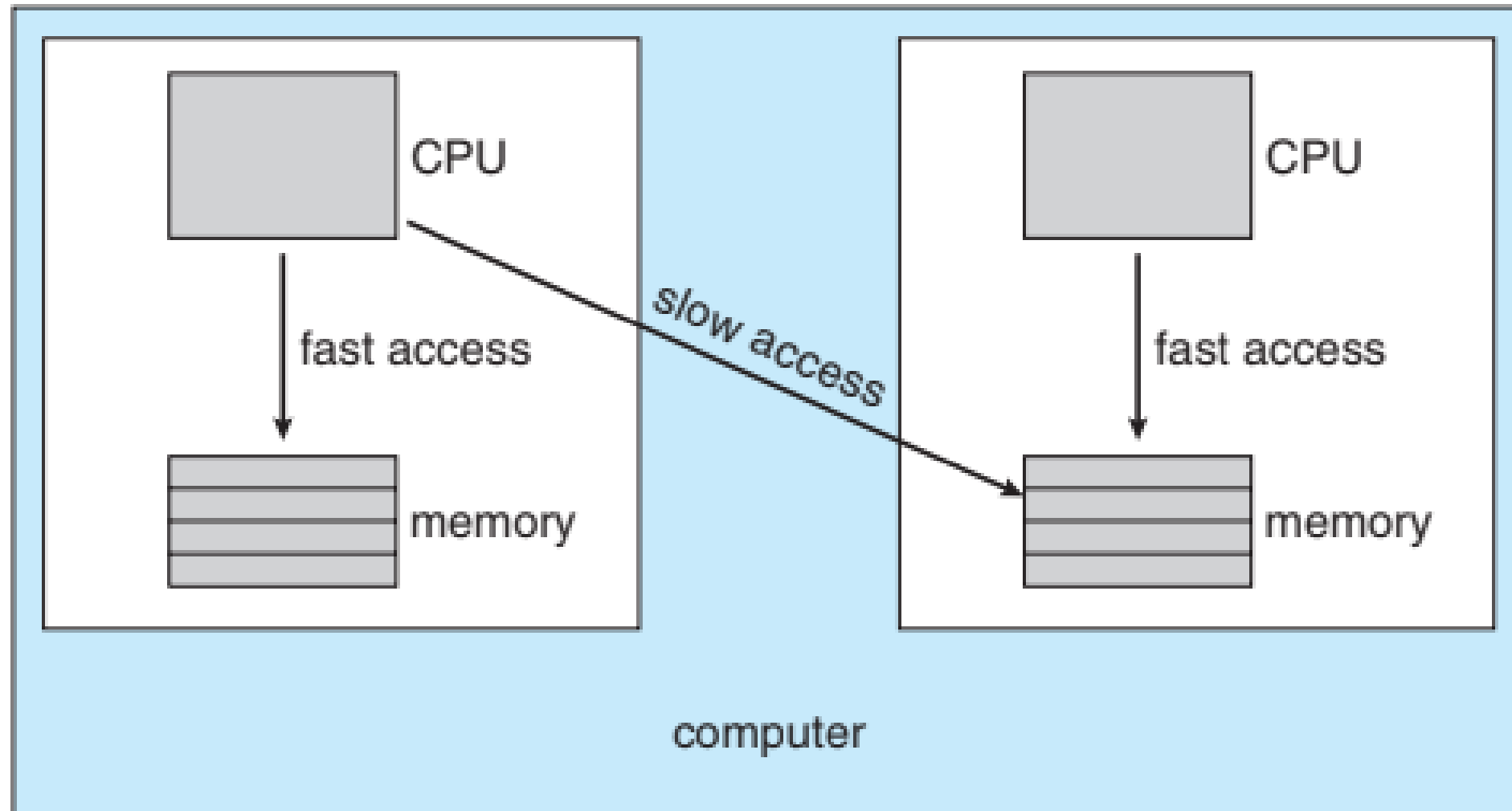


Figure 6.9 NUMA and CPU scheduling.



Load Balancing in SMP Systems

The Goal: On Symmetric Multiprocessing (SMP) systems, it is critical to keep the workload evenly distributed across all processors to fully utilize the hardware.

When is it necessary?

- **Common Queues:** If all CPUs pull from one giant shared queue, load balancing happens naturally.
- **Private Queues:** Load balancing is explicitly required on modern systems where each CPU has its own private queue of eligible processes.

Two Migration Strategies:

- **Push Migration:** A specific background task periodically checks the load on each processor. If it finds an imbalance, it forcibly moves (or pushes) processes from overloaded processors to idle ones.
- **Pull Migration:** An idle processor looks around and actively pulls a waiting task from a busy processor to work on.

The Engineering Conflict: Balancing vs. Affinity

The Tug-of-War: Load balancing actively counteracts the benefits of processor affinity.

The Problem:

- Affinity wants to keep a process running on the same processor so it can use its ultra-fast cache memory.
- Load Balancing wants to push or pull that process to a new processor, which completely destroys the cache memory advantage.

The Compromise: There is no single "correct" rule in systems engineering.

- Some systems prioritize balancing and will always let an idle processor pull a task.
- Other systems prioritize affinity and will only move a process if the load imbalance exceeds a severely high threshold.



The Shift to Multicore: Traditionally, SMP systems used multiple physical processor chips. Today, the standard is placing multiple processor cores on the exact same physical chip.

- **Benefits:** They are significantly faster and consume less power.
- **OS Perspective:** Each core maintains its own architectural state, so the OS still sees them as separate physical processors.

The Hardware Bottleneck: Memory Stalls:

- Processors are incredibly fast, but memory access is comparatively slow.
- When a processor needs to access memory (e.g., a cache miss), it experiences a memory stall.
- During a memory stall, the processor can spend up to 50 percent of its time just sitting idle, waiting for the data to become available.



Memory Stalls

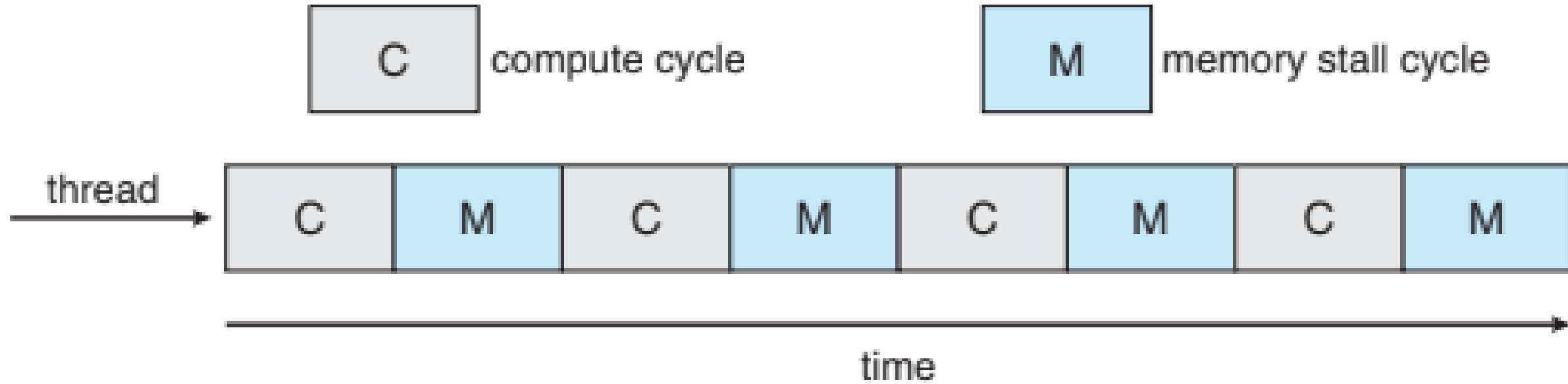


Figure 6.10 Memory stall.



Multithreaded Cores

The Solution: To fix memory stalls, hardware designers implemented multithreaded processor cores. Two or more hardware threads are assigned to each physical core.

- **How it works:** If one thread stalls waiting for memory, the core instantly switches to another thread to keep the processor busy.

Logical Processors: To the OS, every hardware thread appears as a distinct "logical processor".

- **Example:** The UltraSPARC T3 has 16 cores, with 8 hardware threads per core. To the operating system, this looks like 128 logical processors.



Two Switching Strategies:

- **Coarse-Grained:** A thread executes until a long-latency event (like a memory stall) forces a switch. The cost to switch is high because the processor's instruction pipeline must be completely flushed.
- **Fine-Grained (Interleaved):** The core switches between threads at a microscopic level, typically at the boundary of a single instruction cycle. The architecture has built-in logic for this, so the switching cost is very small.

Multithreaded Cores

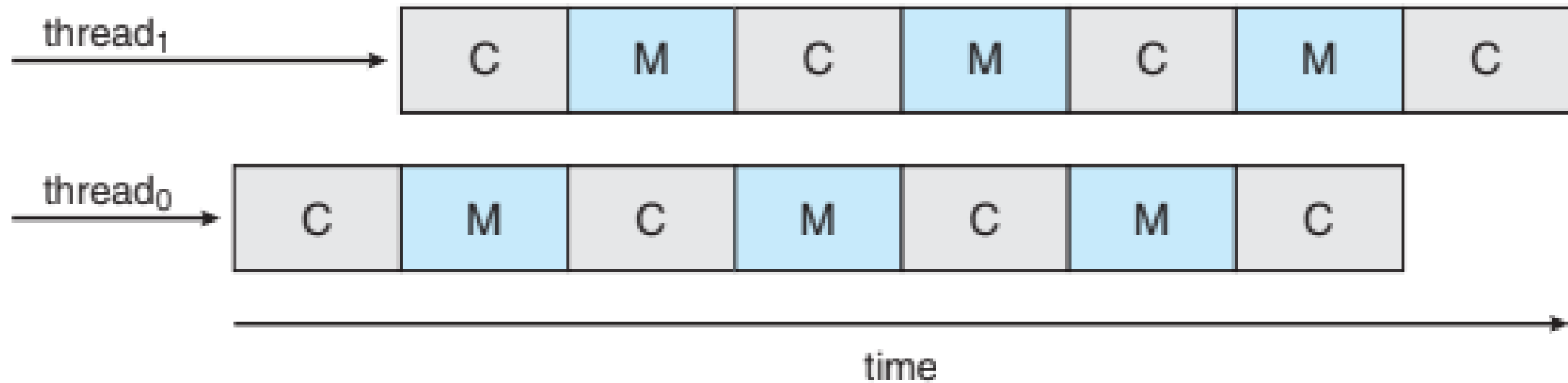


Figure 6.11 Multithreaded multicore system.

A New Complexity: A multithreaded multicore processor creates a scenario where two completely different levels of scheduling must happen simultaneously.

Level 1: The Operating System (Software Level)

- The OS scheduler decides which software thread to run on each available logical processor (hardware thread).
- It uses the standard algorithms we learned previously (e.g., FCFS, SJF, Round-Robin).



Level 2: The Core Hardware (Physical Level)

- The physical core itself must decide which hardware thread actually gets to run its instructions at any given microsecond.
- **Strategy 1 (UltraSPARC T3):** Uses a simple Round-Robin algorithm to juggle its 8 hardware threads.
- **Strategy 2 (Intel Itanium):** Assigns a dynamic "urgency value" (0 to 7) to each thread. When a switching event occurs, the built-in hardware logic simply selects the thread with the highest urgency.



Algorithm Evaluation: How to Choose a Scheduler



The Selection Challenge

The Core Question: With so many algorithms (FCFS, SJF, RR, Priority), how do we pick the right one?

- **No Universal Winner:** Different systems have different goals.
- **Defining Criteria:** We must prioritize specific performance metrics:
 - **CPU Utilization:** Keeping the processor as busy as possible.
 - **Throughput:** Maximizing the number of processes completed per hour.
 - **Response Time:** Minimizing the time between a request and the first response.
- **Example Constraint:** "Maximize throughput while ensuring no process waits more than 2 seconds."



Method 1 - Deterministic Modeling

Definition: A type of Analytic Evaluation that uses a predetermined workload to calculate the performance of each algorithm.

The Process:

- Define a fixed set of processes and their burst times (e.g., $P1=10\text{ms}$, $P2=29\text{ms}$).
- Manually calculate metrics like Average Waiting Time for FCFS, SJF, and RR.

Pros: Simple, fast, and provides exact numbers for comparison.

Cons: Results only apply to that specific workload; it doesn't predict how the system behaves under varied conditions.

Deterministic Example (Case Study)

Workload (All processes arrive at time = 0): P1 = 10 ms, P2 = 29 ms, P3 = 3 ms, P4 = 7 ms, P5 = 12 ms

Algorithm	Average Waiting Time	Performance Rank
FCFS (First Come First Serve)	28 ms	Worst
RR (Round Robin, q = 10)	23 ms	Intermediate
SJF (Shortest Job First)	13 ms	Best

Key Takeaway: For a deterministic workload where all processes arrive simultaneously, Shortest Job First (SJF) is mathematically proven to produce the minimum average waiting time compared to other scheduling algorithms.



- **Definition:** Using mathematical formulas and probability distributions (like the Exponential Distribution) to represent CPU bursts and arrival times.
- **The System as a Network:** The CPU and I/O devices are viewed as "servers" with queues.



Method 2 - Queueing Models

- **Little's Formula:** A fundamental law in queueing theory.

$$n = \lambda \times W$$

- n : Average queue length.
 - λ : Average arrival rate (e.g., 3 processes/sec).
 - W : Average waiting time in the queue.
- **Use Case:** If we know 7 processes arrive per second and 14 are usually in the queue, we can calculate that the average wait is **2 seconds**.

Strengths:

- Valid for any scheduling algorithm and arrival distribution.
- Useful for high-level "steady state" analysis.

Weaknesses:

- Complex algorithms are mathematically difficult to model.
- Often requires "simplifying assumptions" that may not reflect reality.
- Distributions used (like Poisson) are often just approximations.

Method 3 - Simulations

Definition: Programming a software model of the computer system to gather performance statistics.

How it Works:

- A Clock variable tracks simulated time.
- Random-Number Generators create processes and burst times based on specific distributions.

The "Trace Tape" Method: To improve accuracy, record actual events from a real system (CPU/IO bursts) and play them back through the simulator.

Why? It accounts for the relationship between successive events that random numbers miss.

Method 4 - Implementation

The "Gold Standard": Code the algorithm directly into the OS and test it in the real world.

The Challenges:

1. **High Cost:** Coding and kernel modification are labor-intensive.
2. **User Frustration:** Users don't like their OS changing constantly while they try to work.
3. **The "Gaming" Factor:** Users may change their behavior to exploit the scheduler (e.g., adding fake I/O to stay in a high-priority queue).

Modern Solution: Flexible scheduling via APIs (Java, Windows, POSIX) or system commands (like `dispadm` in Solaris) to tune parameters without rewriting the kernel.